Iteradores y algoritmos de la STL

Qué es de verdad `v.begin()`

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Septiembre 2026

Objetivos de aprendizaje I

Al finalizar esta sesión, el estudiante podrá

- Explicar qué es un iterador, recorrer con él un `vector` y un `string`, y pasar de posición a índice y de índice a posición.
- Reconocer cuándo un iterador deja de valer tras `insert` o `erase`, y escribir el recorrido que borra mientras avanza sin caer en ese error.
- Aplicar `lower_bound`, `unique`, `accumulate` y `find` sobre objetos, y escoger entre ellos por lo que cuestan.
- Traducir los ejercicios de los contratos a la biblioteca: postfija con `stack`, intersección con dos iteradores, frecuencias y mediana con `sort`.
- Decir qué cuesta en tiempo y en espacio cada operación de un iterador y cada algoritmo de la sesión.

Objetivos de aprendizaje II

Objetivos de aprendizaje del curso

- OA1: implementar soluciones a problemas pequeños con las estructuras que provee el lenguaje.
- OA2: escoger un contenedor y un algoritmo por el costo de sus operaciones.
- OA3: leer documentación en inglés y traducirla a un contrato con su costo.

Referencias

B. Stroustrup, *The C++ Programming Language*, 4.ª ed., 2013, capítulo 33 (iteradores) y capítulo 32 (algoritmos). Documentación en línea: `https://en.cppreference.com/w/cpp/iterator`.

Contenido I

1 Lo que dejó la STL

2 Un iterador es un puntero que sabe recorrer

3 El iterador colgante

4 Los algoritmos que faltaban

5 Ejercicios con la biblioteca

6 Para el juez

7 Lo que cobra cada operación

Contenido I

1 Lo que dejó la STL

2 Un iterador es un puntero que sabe recorrer

3 El iterador colgante

4 Los algoritmos que faltaban

5 Ejercicios con la biblioteca

6 Para el juez

7 Lo que cobra cada operación

Cuatro contenedores, cinco algoritmos y una expresión sin explicar I

Lo que quedó

- `vector`, `string`, `stack` y `queue`: los contratos del curso con los nombres de la biblioteca, y `at` vigilando lo que `[]` no.
- `sort`, `reverse`, `count`, `find`, `binary_search`: reciben un rango `[begin, end)` y `sort` acepta un criterio propio.
- Todo nombre de la biblioteca vive en `std` y se trae con `using std::nombre`.

```
v.insert(v.begin() + 1, 7);
v.erase(v.begin() + 0);
sort(v.begin(), v.end());
int menor = *min_element(v.begin(), v.end());
```

Cuatro contenedores, cinco algoritmos y una expresión sin explicar II

Pregunta

`v.begin()` apareció cuatro veces y se usó como "la posición 0". ¿Qué es? ¿Por qué se le puede sumar 1? ¿Y qué es el `*` adelante de `min_element`?

Contenido I

1 Lo que dejó la STL

2 Un iterador es un puntero que sabe recorrer

3 El iterador colgante

4 Los algoritmos que faltaban

5 Ejercicios con la biblioteca

6 Para el juez

7 Lo que cobra cada operación

Lo que ya sabían hacer con un arreglo I

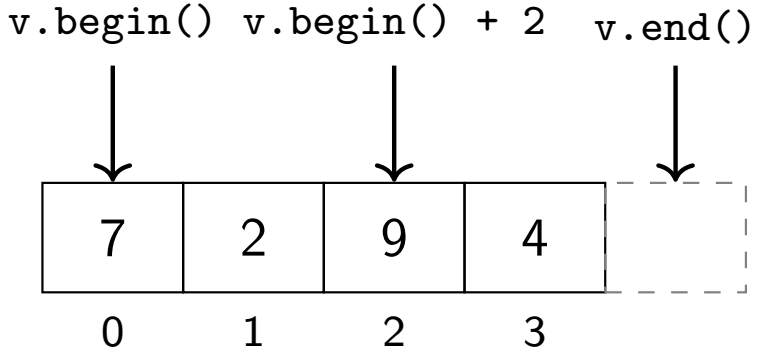
```
int a[5] = {9, 1, 8, 3, 5};
int *p = a;              // la direccion de la primera casilla
while (p != a + 5) {     // a + 5: justo despues de la ultima
  printf(" %d", *p);     // lo que hay en esa casilla
  ++p;                   // la siguiente casilla
}
```

- Es el recorrido con punteros de la sesión de arreglos: una dirección que se lee con *, se avanza con ++ y se compara con la dirección que sigue a la última casilla.
- Ese puntero sabe tres cosas: dónde está, qué hay ahí y cuál es la siguiente. Eso, y nada más, es un iterador.

Lo que ya sabían hacer con un arreglo II

Definición (Iterador)

Un objeto que señala un elemento dentro de un contenedor y sabe pasar al siguiente. Se lee con `*it`, se avanza con `++it` y se compara con == y !=. Cada contenedor define el suyo: `vector<int>::iterator`, `string::iterator`. Stroustrup, capítulo 33.



- `v.end()` señala *justo después* del último, como `a + 5`: no se lee, solo sirve para saber que el recorrido terminó.

El mismo recorrido, dos veces I

```
void imprimirIndice(vector<int> &v) {
  int p = 0;
  while (p < (int) v.size()) {
    printf(" %d", v[p]);
    p = p + 1;
  }
  printf("\n");
}

void imprimirIterador(vector<int> &v) {
  vector<int>::iterator it = v.begin();
  while (it != v.end()) {
    printf(" %d", *it);
    ++it;
  }
  printf("\n");
}
```

El mismo recorrido, dos veces II

con puntero sobre arreglo	con índice	con iterador
`int *p = a`	`int p = 0`	`it = v.begin()`
`p != a + n`	`p < n`	`it != v.end()`
`*p`	`v[p]`	`*it`
`++p`	`p = p + 1`	`++it`
`p - a`	`p`	`it - v.begin()`
`a + k`	`k`	`v.begin() + k`

- Salida real de las dos: `7 2 9 4`. Con `v = {7, 2, 9, 4}`, `*(v.begin() + 2)` es 9 y `(v.begin() + 2) - v.begin()` es 2: restar `v.begin()` convierte una posición en índice.
- En `vector` las dos formas cuestan lo mismo, $O(1)$ por paso. La diferencia llega con los contenedores que no tienen índice: `list`, `map`, `set`. Ahí solo existe la segunda forma, y por eso se aprende ahora.

Una cadena y un arreglo de C, con el mismo recorrido I

```
int vocales(string &s) {
  int c = 0;
  string::iterator it = s.begin();
  while (it != s.end()) {
    if (*it == 'a' || *it == 'e' || *it == 'i'
        || *it == 'o' || *it == 'u') {
      c = c + 1;
    }
    ++it;
  }
  return c;
}
```

```
int a[5] = {9, 1, 8, 3, 5};
sort(a, a + 5);              // un puntero es un iterador
```

Una cadena y un arreglo de C, con el mismo recorrido II

- Salida real: `vocales de iterador: 4` y, tras el `sort`, `1 3 5 8 9`.
- Los algoritmos de `<algorithm>` piden un rango de iteradores, y un par de punteros a un arreglo lo es. La biblioteca no distingue: por eso el mismo `sort` ordena un `vector`, un `string` o un arreglo de C.

Costos de un iterador de `vector` o `string`

`begin()`, `end()`, `*it`, `++it`, `it + k` e `it - v.begin()` cuestan $O(1)$: son sumas de direcciones, como con el puntero. Un recorrido completo cuesta $\Theta(n)$ y $\Theta(1)$ de espacio aparte del contenedor.

Contenido I

1 Lo que dejó la STL

2 Un iterador es un puntero que sabe recorrer

3 El iterador colgante

4 Los algoritmos que faltaban

5 Ejercicios con la biblioteca

6 Para el juez

7 Lo que cobra cada operación

Un iterador guardado antes de insert I

```
vector<int> v = {7, 2, 9, 4};
vector<int>::iterator it = v.begin() + 1;   // senala el 2
v.insert(v.begin(), 5);
printf("%d\n", *it);                        // ?
```

Pregunta

¿Qué imprime? El 2 se corrió a la posición 2, y puede que el vector entero se haya mudado a otra zona de memoria para crecer.

Un iterador guardado antes de insert II

No está definido

Tras `insert` o `erase`, los iteradores guardados dejan de valer: señalan una casilla que ya no es la que era, o memoria que el vector devolvió. Es el puntero colgante de la sesión de memoria dinámica, con otro nombre.

```
it = v.insert(v.begin(), 5);    // devuelve la posicion del 5
it = v.begin() + 2;             // se vuelve a tomar: el 2
it = v.erase(it);               // devuelve la posicion
↪  siguiente
```

- La regla: tras `insert` o `erase` se toma el iterador de nuevo, y las dos lo devuelven para eso. Salida real: `*it = 5, indice 0`; `*it = 9, indice 2`; el vector queda `5 7 9 4`.

Borrar mientras se recorre I

Problema

Quitar todos los ceros de $\{0, 3, 0, 0, 8, 1, 0\}$.

```
void quitarCerosErase(vector<int> &v) {
  vector<int>::iterator it = v.begin();
  while (it != v.end()) {
    if (*it == 0) {
      it = v.erase(it);   // el siguiente ya queda en it
    } else {
      ++it;               // se avanza solo si no se borro
    }
  }
}
```

- El error productivo: `v.erase(it)` y luego `++it` siempre. Tras el `erase`, `it` no vale y el `++` avanza sobre nada.
- Costo: cada `erase` en la posición p corre $n - p$ elementos; con k ceros, $O(kn)$, y $\Theta(n^2)$ si la mitad son ceros.

Borrar mientras se recorre II

```
void quitarCeros(vector<int> &v) {
  int destino = 0;
  int p = 0;
  while (p < (int) v.size()) {
    if (v[p] != 0) {
      v[destino] = v[p];      // se copia hacia el frente
      destino = destino + 1;
    }
    p = p + 1;
  }
  v.erase(v.begin() + destino, v.end());   // se recorta la cola
}
```

- La misma respuesta, 3 8 1, en un recorrido: $\Theta(n)$ de tiempo y $\Theta(1)$ de espacio; el `erase` de un rango al final no corre nada. Las dos versiones quedan: una dice cómo se borra con seguridad, la otra cuánto cuesta no correr los elementos una vez por cero.

Contenido I

1 Lo que dejó la STL

2 Un iterador es un puntero que sabe recorrer

3 El iterador colgante

4 Los algoritmos que faltaban

5 Ejercicios con la biblioteca

6 Para el juez

7 Lo que cobra cada operación

lower_bound: la posición que binary_search no daba I

```cpp
vector<int> v = {5, 2, 3, 5, 1, 5, 8};
sort(v.begin(), v.end());                    // < 1 2 3 5 5 5 8 >
vector<int>::iterator it = lower_bound(v.begin(), v.end(), 5);
int pos = (int) (it - v.begin());
if (it != v.end() && *it == 5) {
  printf("5 esta, la primera vez en la posicion %d\n", pos);
} else {
  printf("5 no esta; iria en la posicion %d\n", pos);
}
```

Contrato

Sobre un rango ordenado, `lower_bound` devuelve la primera posición cuyo valor **no es menor** que x: la primera aparición de x si está, o el sitio donde iría si no está. Parte el rango por la mitad cada vez, $O(\log n)$, y exige orden igual que `binary_search`.

lower_bound: la posición que binary_search no daba II

Salida real

```
5 esta, la primera vez en la posicion 3
4 no esta; iria en la posicion 3
9 no esta; iria en la posicion 7
1 esta, la primera vez en la posicion 0
```

- Las dos comprobaciones son necesarias: si x es mayor que todos, `it` es `v.end()` y leer `*it` es el error colgante de la sección anterior. Con 9 el índice es 7, el tamaño del vector.
- `find` responde lo mismo sin exigir orden, en $\Theta(n)$. Con muchas consultas sobre el mismo vector, ordenar una vez ($O(n \log n)$) y preguntar con `lower_bound` ($O(\log n)$ cada una) le gana a `find` desde la segunda consulta.

unique: quitar repetidos en dos pasos I

```
vector<int> v = {4, 1, 4, 9, 1, 1, 4};
sort(v.begin(), v.end());                          // < 1 1 1 4
↪  4 4 9 >
vector<int>::iterator fin = unique(v.begin(), v.end());
v.erase(fin, v.end());                             // < 1 4 9 >
```

Lo que deja cada paso

```
1 1 1 4 4 4 9        tras sort
1 4 9 4 4 4 9        tras unique: 3 distintos al frente
1 4 9                tras erase
```

- `unique` no borra: copia hacia el frente cada valor distinto del anterior, como `quitarCeros`, y devuelve la posición donde termina lo válido. Lo que queda después no significa nada, y por eso el `erase` cierra el paso.

unique: quitar repetidos en dos pasos II

- Solo junta repetidos **consecutivos**: sin el `sort` previo, {`4, 1, 4`} quedaría igual.
- Costo: $O(n \log n)$ del `sort` más $\Theta(n)$ de `unique` y $O(1)$ del `erase` al final. El `quitarRepetidos` de la sesión de ejercicios, con `eliminar` en medio, era $O(n^2)$; este exige que el orden original no importe.

accumulate: la suma y el tipo de la suma I

```
vector<int> v(3, 1000000000);              // tres veces 10^9
int chico = accumulate(v.begin(), v.end(), 0);
long long grande = accumulate(v.begin(), v.end(), 0LL);
```

Salida real

```
con 0:   -1294967296
con 0LL: 3000000000
```

- `accumulate`, de `<numeric>`, suma el rango partiendo del valor inicial, cuyo tipo es el tipo de la suma: con `0` se acumula en `int` y $3 \cdot 10^9$ no cabe; con `0LL`, en `long long`. El contenedor puede ser de `int` y la suma no. $\Theta(n)$ en tiempo, $\Theta(1)$ de espacio.

find sobre objetos I

```
class Estudiante {
public:
  string nombre;
  double nota;
  Estudiante(string n, double x) { nombre = n; nota = x; }
  // el mismo estudiante: el mismo nombre
  bool operator==(const Estudiante &otro) const {
    return nombre == otro.nombre;
  }
};

vector<Estudiante>::iterator it =
    find(g.begin(), g.end(), Estudiante("Ana", 0));
if (it != g.end()) {
  printf("Ana esta en la posicion %d con nota %.1f\n",
         (int) (it - g.begin()), it->nota);
}
```

find sobre objetos II

- `find` compara con ==, y con una clase propia ese == lo escribe la clase, como el + de `Racional` en la sesión de abstracción. Aquí igualdad es mismo nombre; la nota del buscado no cuenta.
- `it->nota` es `(*it).nota`: el iterador se usa como un puntero a objeto. Salida real: `Ana esta en la posicion 2 con nota 3.8 y Juan no esta`.

Contenido I

1 Lo que dejó la STL

2 Un iterador es un puntero que sabe recorrer

3 El iterador colgante

4 Los algoritmos que faltaban

5 Ejercicios con la biblioteca

6 Para el juez

7 Lo que cobra cada operación

Ejercicio en parejas: la intersección con dos iteradores I

Enunciado

Traduzca `interseccion` de la sesión de ejercicios, que recibía dos listas ordenadas y devolvía los comunes, a dos `vector<int>` ordenados. Con {`1, 3, 4, 7, 9`} y {`2, 3, 7, 8, 9`} debe dar `3 7 9`, y costar lo mismo que con las listas: $\Theta(n_a + n_b)$ en tiempo y $O(\text{mín}(n_a, n_b))$ de espacio para el resultado.

Pistas

- Con las listas eran dos índices que avanzaban; con iteradores es lo mismo, cambiando `obtener(i)` por `*i` y `i + 1` por `++i`.
- ¿Cuándo termina? Cuando cualquiera de los dos llega a su `end()`.

Ejercicio en parejas: la intersección con dos iteradores II

Una solución

```
vector<int> interseccion(vector<int> &a, vector<int> &b) {
  vector<int> r;
  vector<int>::iterator i = a.begin();
  vector<int>::iterator j = b.begin();
  while (i != a.end() && j != b.end()) {
    if (*i < *j) {
      ++i;
    } else if (*j < *i) {
      ++j;
    } else {
      r.push_back(*i);
      ++i;
      ++j;
    }
  }
  return r;
}
```

Ejercicio en parejas: frecuencias con sort I

Enunciado

Dado `vector<string> p = {çola", "pila", çola", "lista", çola", "pila"}`, imprima cada palabra distinta con el número de veces que aparece, en orden alfabético.

Pistas

- Ordene primero. Después, los iguales quedan juntos y el conteo es un solo recorrido, como en `unique`.
- Dos ciclos anidados sobre el mismo iterador: el de afuera toma una palabra, el de adentro cuenta mientras se repita.

Ejercicio en parejas: frecuencias con sort II

Una solución

```
sort(p.begin(), p.end());
vector<string>::iterator it = p.begin();
while (it != p.end()) {
  string actual = *it;
  int cuenta = 0;
  while (it != p.end() && *it == actual) {
    cuenta = cuenta + 1;
    ++it;
  }
  printf("%s %d\n", actual.c_str(), cuenta);
}
```

- Salida real: `cola 3`, `lista 1`, `pila 2`. Cada elemento se visita una vez entre los dos ciclos: $\Theta(n)$ tras el `sort`, $O(n \log n)$ en total y $\Theta(1)$ de espacio aparte. Sin ordenar, buscando cada palabra en las anteriores, sería $O(n^2)$.

La mediana con sort I

```
double mediana(vector<int> v) {     // por valor: ordena una copia
  sort(v.begin(), v.end());
  int n = (int) v.size();
  double m = 0;
  if (n % 2 == 1) {
    m = v[n / 2];
  } else {
    m = (v[n / 2 - 1] + v[n / 2]) / 2.0;
  }
  return m;
}
```

- Salida real: `5.0` con $\{$`9, 1, 7, 3, 5`$\}$ y `5.5` con $\{$`9, 1, 7, 4`$\}$; a sigue como estaba. Por valor a propósito: el llamador conserva su orden y paga la copia, $\Theta(n)$ de espacio, más el `sort`, $O(n \log n)$. Sin ordenar todo se puede en $\Theta(n)$ (`nth_element`), tema de un curso posterior.

La postfija con stack: push, top y pop I

```
// el evaluar de los ejercicios: "3 4 + 2 *" da 14 y "8 2 - 3 *" da
↪  18
int evaluar(const char *s) {
  stack<int> p;
  int i = 0;
  while (s[i] != '\0') {
    if (s[i] >= '0' && s[i] <= '9') {
      p.push(s[i] - '0');
    } else if (s[i] != ' ') {
      int b = p.top();
      p.pop();
      int a = p.top();
      p.pop();
      if (s[i] == '+') { p.push(a + b); }
      else if (s[i] == '-') { p.push(a - b); }
      else { p.push(a * b); }
    }
    i = i + 1;
  }
  return p.top();
}
```

Contenido I

1 Lo que dejó la STL

2 Un iterador es un puntero que sabe recorrer

3 El iterador colgante

4 Los algoritmos que faltaban

5 Ejercicios con la biblioteca

6 Para el juez

7 Lo que cobra cada operación

Para el juez: UVa 10474 — Where is the Marble? I

`https://vjudge.net/problem/UVA-10474`
Envío: `https://vjudge.net/problem/UVA-10474`

Qué pide

N canicas numeradas y Q consultas. Para cada consulta x, decir en qué posición (contando desde 1) está la **primera** canica con ese número una vez ordenadas, o que no está.

- **Entrada.** Casos hasta $N = 0$ y $Q = 0$. Cada caso: una línea con N y Q, luego N números (sin orden) y Q consultas. Nada supera 10 000 y hasta 65 casos.
- **Salida.** `CASE# k:` y luego, por consulta, `x found at y` o `x not found`.

Para el juez: UVa 10474 — Where is the Marble? II

- **Con la sesión.** `sort` y luego `lower_bound` por consulta: la primera aparición es exactamente lo que devuelve. Posición $=$ `it - v.begin()` $+ 1$. Costo por caso $O(N \log N + Q \log N)$.
- **La trampa.** Los dos chequeos de `lower_bound`: que no sea `end()` y que el valor sea x. Y vaciar el vector entre casos.

Para el juez: UVa 10474 — Where is the Marble? III

Sample Input

```
4 1
2
3
5
1
5
5 2
1
3
3
3
1
2
3
0 0
```

Sample Output

```
CASE# 1:
5 found at 4
CASE# 2:
2 not found
3 found at 3
```

Para el juez: UVa 11849 — CD I

`https://vjudge.net/problem/UVA-11849`
Envío: `https://vjudge.net/problem/UVA-11849`

Qué pide

Jack tiene N discos y Jill M, cada lista en orden creciente y sin repetidos. ¿Cuántos títulos tienen los dos?

- **Entrada.** Casos hasta `0 0`. Cada caso: N y M (hasta un millón cada uno), luego N números y M números, ya ordenados, cada uno hasta 10^9.
- **Salida.** Un entero por caso.
- **Con la sesión.** Es `interseccion` contando en vez de guardando: dos iteradores que avanzan, $\Theta(N + M)$. Como las listas ya vienen ordenadas, no hay `sort`.

Para el juez: UVa 11849 — CD II

- **La trampa.** Hasta dos millones de números por caso: leer con `scanf`, no con `cin`, y no buscar cada disco de Jack en los de Jill con `find` ($O(NM)$) ni con `binary_search` ($O(N \log M)$, que pasa pero es más lento que avanzar los dos).

Para el juez: UVa 11849 — CD III

Sample Input

```
3 3
1
2
3
1
2
4
0 0
```

Sample Output

```
2
```

Para el juez: UVa 10008 — What's Cryptanalysis? I

`https://vjudge.net/problem/UVA-10008`
Envío: `https://vjudge.net/problem/UVA-10008`

Qué pide

Contar cuántas veces aparece cada letra en un texto, sin distinguir mayúsculas, e imprimir las letras que aparecen de mayor a menor frecuencia; en empate, en orden alfabético.

- **Entrada.** Una línea con n y luego n líneas de texto, con espacios y signos, que se leen con `fgets`.
- **Salida.** Por cada letra que aparece, la mayúscula, un espacio y su cuenta.

Para el juez: UVa 10008 — What's Cryptanalysis? II

- **Con la sesión.** Un arreglo de 26 cuentas; después, un `vector` de objetos {`letra, cuenta`} y `sort` con un criterio `antes`: cuenta de mayor a menor y, en empate, letra de menor a mayor, con $<$ estricto. Es el `antes` de los estudiantes con otros dos campos.
- **La trampa.** Solo letras: `isalpha` decide y `toupper` unifica. Una letra con cuenta 0 no se imprime.

Para el juez: UVa 10008 — What's Cryptanalysis? III

Sample Input

```
3
This is a test.
Count me 1 2 3 4 5.
Wow!!!! Is this question easy?
```

Sample Output

```
S 7
T 6
I 5
E 4
O 3
A 2
H 2
N 2
U 2
W 2
C 1
M 1
Q 1
Y 1
```

Contenido I

1 Lo que dejó la STL

2 Un iterador es un puntero que sabe recorrer

3 El iterador colgante

4 Los algoritmos que faltaban

5 Ejercicios con la biblioteca

6 Para el juez

7 Lo que cobra cada operación

Lo que cobra cada operación I

Operación	Tiempo	Espacio aparte del contenedor
`begin`, `end`, `*it`, `++it`, `it + k`, `it - begin`	$O(1)$	$O(1)$
recorrido completo con iterador	$\Theta(n)$	$\Theta(1)$
`insert`, `erase` en la posición p	$O(n-p)$	$O(1)$
`erase` de un rango al final	$O(1)$	$O(1)$
borrar k elementos con `it = erase(it)`	$O(kn)$	$O(1)$
borrar k elementos compactando	$\Theta(n)$	$O(1)$
`lower_bound` sobre orden	$O(\log n)$	$O(1)$
`unique`	$\Theta(n)$	$O(1)$
`accumulate`	$\Theta(n)$	$O(1)$
`find` sobre objetos	$\Theta(n)$ comparaciones con ==	$O(1)$
`sort`	$O(n \log n)$	$O(\log n)$, la recursión
`interseccion` con dos iteradores	$\Theta(n_a + n_b)$	$O(\text{mín}(n_a, n_b))$, el resultado
frecuencias con `sort`	$O(n \log n)$	$O(1)$
`mediana` por valor	$O(n \log n)$	$\Theta(n)$, la copia
`evaluar` con `stack`	$\Theta(n)$	$O(n)$, la pila

Lo que cobra cada operación II

- Las operaciones del iterador son aritmética de direcciones, por eso $O(1)$; los algoritmos cobran lo que recorren y `lower_bound` parte por la mitad, por eso exige orden. Dos rutas al mismo resultado con costo distinto es lo que se escoge al programar: `erase` uno a uno o compactar, `find` por consulta u ordenar una vez y `lower_bound`.

Propuestos para la casa I

1. **Al revés con iteradores.** Escriba `alReves` recorriendo el `string` desde `s.end() - 1` hacia `s.begin()`. ¿Cuál es la condición de parada?
2. **Un iterador que sobrevive.** Con `{7, 2, 9, 4}` e `it = v.begin() + 3`, ¿cuáles de `v[0] = 1`, `sort`, `push_back` y `erase(v.begin())` lo dejan válido?
3. **Todas las posiciones.** `upper_bound` da la posición siguiente a la última aparición de x; con `lower_bound` y él, cuente cuántas veces está x en $O(\log n)$.
4. **Frecuencias sin ordenar.** Cuente las palabras buscando cada una en las ya contadas con `find`. ¿Cuántas comparaciones hace con 6 palabras y con 60?
5. **La mediana en Python.** Escríbala con `sorted`. ¿Qué operación paga la copia que en C++ se pagó al recibir por valor?

Resumen I

- Un iterador es un puntero que sabe recorrer: `*it`, `++it`, `!= end()`; sirve igual para `vector`, `string` y, cuando lleguen, `list`, `map` y `set`. `v.begin() + k` es la posición k e `it - v.begin()` el índice.
- Tras `insert` o `erase` el iterador guardado no vale: se toma el que devuelven. Borrar mientras se recorre es `it = v.erase(it)` o, mejor, compactar.
- `lower_bound` da la posición sobre orden en $O(\log n)$; `unique` junta repetidos consecutivos; `accumulate` suma en el tipo del valor inicial; `find` sobre objetos usa el `==` de la clase.

Lo que sigue

Las implementaciones: cómo se construye por dentro la lista estática que la biblioteca ya trae, y con ella el primer algoritmo de ordenamiento, insertion sort.

Referencias I


B. Stroustrup. *The C++ Programming Language*, 4.ª ed. Addison-Wesley, 2013. Capítulo 33, iteradores; capítulo 32, algoritmos de la STL.


cppreference. `std::vector::iterator`, `std::lower_bound`, `std::unique`, `std::accumulate`. `https://en.cppreference.com/w/cpp/iterator`, `https://en.cppreference.com/w/cpp/algorithm/lower_bound`, `https://en.cppreference.com/w/cpp/algorithm/unique`, `https://en.cppreference.com/w/cpp/algorithm/accumulate`.


UVa Online Judge, problemas 10474, 11849 y 10008. `https://vjudge.net`.